
Author: The Tuan Trinh
Date: 2026-05-22

```
In [1]: import os
import platform
print("=" * 100)
print("OS Name:", os.name)
print("System:", platform.system())
print("Release:", platform.release())
print("Machine:", platform.machine())
print("Processor:", platform.processor())
print("Platform:", platform.platform())

print("\nEnvironment Variables (sample):")
for key, value in list(os.environ.items())[:15]:
    print(f"{key} = {value}")
```

```
=====
OS Name: posix
System: Linux
Release: 4.18.0-348.12.2.el8_5.x86_64
Machine: x86_64
Processor: x86_64
Platform: Linux-4.18.0-348.12.2.el8_5.x86_64-x86_64-with-glibc2.28
```

```
Environment Variables (sample):
CONDA_SHLVL = 2
LD_LIBRARY_PATH = /usr/local/cuda/lib64:/usr/local/cuda/lib64:
CONDA_EXE = /work/tuan.tt19010226/miniconda/bin/conda
SSH_CONNECTION = 10.4.98.68 64873 10.20.8.11 22
CONDA_PREFIX = /work/tuan.tt19010226/conda/envs/RadarLM
XDG_SESSION_ID = 64511
USER = tuan.tt19010226
PWD = /home/tuan.tt19010226
HOME = /home/tuan.tt19010226
CONDA_PYTHON_EXE = /work/tuan.tt19010226/miniconda/bin/python
SSH_CLIENT = 10.4.98.68 64873 22
MESA_LOADER_DRIVER_OVERRIDE = llvmpipeexport
CONDA_PROMPT_MODIFIER = (RadarLM)
VSCODE_CLI_REQUIRE_TOKEN = b7ebbf12-935b-4329-b0c7-398a25100c8f
SHELL = /bin/bash
```

```
In [2]: platform.python_version()
```

```
Out[2]: '3.10.19'
```

Lab 2 - Arithmetic Operations on Images

Image Addition

```
In [3]: import cv2
import numpy as np
x = np.uint8([250])
y = np.uint8([10])

print(cv2.add(x,y))
```

```
[[260.]
 [  0.]
 [  0.]
 [  0.]]
```

```
In [4]: print(x+y)
```

```
[4]
```

Image Blending

This is also image addition, but different weights are given to images in order to give a feeling of blending or transparency. Images are added as per the equation below:

$$g(x) = (1 - \alpha)f_0(x) + \alpha f_1(x)$$

By varying α from $0 \rightarrow 1$, you can perform a cool transition between one image to another.

Here I took two images to blend together. The first image is given a weight of 0.7 and the second image is given 0.3. `cv.addWeighted()` applies the following equation to the image:

$$dst = \alpha \cdot img1 + \beta \cdot img2 + \gamma$$

Here γ is taken as zero.

```
In [5]: import cv2
import urllib.request
import numpy as np
import matplotlib.pyplot as plt

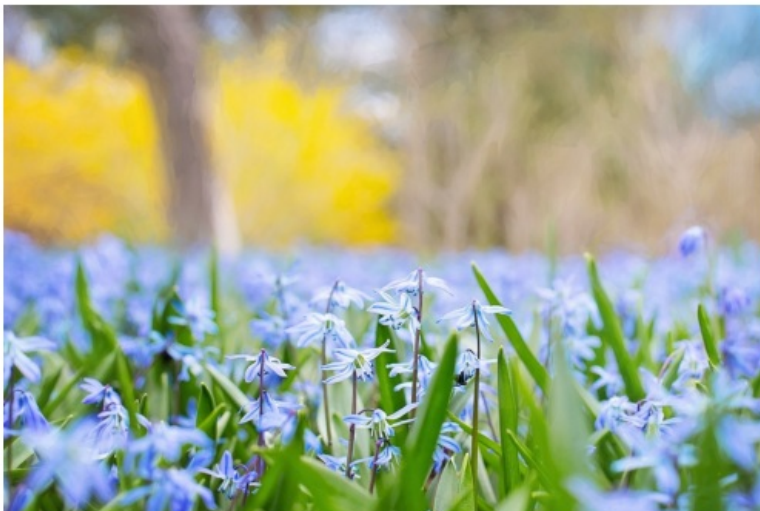
url = "https://64.media.tumblr.com/4e4be46ca4a62f770797c31f4dd91088/tumblr_pn8bq6ZvGc1tawn8uo1_1280.jpg"

resp = urllib.request.urlopen(url)
img = np.asarray(bytearray(resp.read()), dtype=np.uint8)
img = cv2.imdecode(img, cv2.IMREAD_COLOR)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img_rgb)
plt.axis(False)
plt.show()
```



```
In [6]: url_ = "https://64.media.tumblr.com/3af477c92719b553a5f74f2319ee1208/tumblr_pq8jvbK4gq1tawn8uo1_1280.jpg"

resp_ = urllib.request.urlopen(url_)
img_1 = np.asarray(bytearray(resp_.read()), dtype=np.uint8)
img_1 = cv2.imdecode(img_1, cv2.IMREAD_COLOR)
img_1 = cv2.cvtColor(img_1, cv2.COLOR_BGR2RGB)
plt.imshow(img_1)
plt.axis(False)
plt.show()
```



```
In [7]: dst = cv2.addWeighted(img_rgb,0.7,img_1,0.3,0)
plt.imshow(dst)
plt.axis(False)
plt.show()
```



Bitwise Operations

```
In [8]: url__ = "https://th.bing.com/th/id/ODF.yf34kah7rom5d3lzVLg1Cw?w=32&h=32&q1t=90&pcl=ffffffa&o=6&pid=1.2"

resp__ = urllib.request.urlopen(url__)
image__ = np.asarray(bytearray(resp__.read()), dtype=np.uint8)
img_2 = cv2.imdecode(image__, cv2.IMREAD_COLOR)
img_2 = cv2.cvtColor(img_2, cv2.COLOR_BGR2RGB)
img_2 = cv2.resize(img_2, (224, 224))
gray = cv2.cvtColor(img_2, cv2.COLOR_RGB2GRAY)
_, mask = cv2.threshold(gray, 240, 255, cv2.THRESH_BINARY)
img_2[mask == 255] = [0, 0, 0]
```

```
In [9]: rows,cols,channels = img_2.shape
roi = dst[0:rows, 0:cols]

img2gray = cv2.cvtColor(img_2,cv2.COLOR_BGR2GRAY)
ret, mask = cv2.threshold(img2gray, 10, 255, cv2.THRESH_BINARY)
mask_inv = cv2.bitwise_not(mask)

img1_bg = cv2.bitwise_and(roi,roi,mask = mask_inv)
img2_fg = cv2.bitwise_and(img_2,img_2,mask = mask)
dst_ = cv2.add(img1_bg,img2_fg)
dst[0:rows, 0:cols ] = dst_
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.imshow(img1_bg)
plt.axis(False)
plt.subplot(1, 2, 2)
plt.imshow(dst)
plt.axis(False)
plt.show()
```

